

Tópico 05 - Parte 2: Mural de Contatos Persistente

UC2 – Banco de Dados Relacional

Prof. Jeronimo Silva

Roteiro da Aula

- **Bloco 1:** Checklist de Conceitos da Aula
- **Bloco 2:** Memória Volátil (UC1) vs Persistência SQLite (UC2)
- **Bloco 3:** Arquitetura em 4 Camadas (`conexao` ↔ `main` ↔ `preload` ↔ `renderer`)
- **Bloco 4:** Canais IPC & Sanitização com Regex
- **Bloco 5:** Roteiro da Prática Guiada (Code-Along)
- **Bloco 6:** Desafio Prático & Encerramento



Checklist de Conceitos da Aula

- IPC (`ipcRenderer.invoke` / `ipcMain.handle`): Comunicação assíncrona bidirecional segura.
- `better-sqlite3` (`prepare` , `.all()` , `.run()`): Execução síncrona e de alta performance no Node.js.
- `AUTOINCREMENT` : Geração automática de ID numérico sequencial único no banco.
- `UNIQUE` : Restrição SQL que previne cadastros duplicados (ex: e-mail).
- Sanitização com Regex (`replace(/\\D/g, '')`): Remoção de caracteres não numéricos.
- `confirm()` : Modal nativo do navegador para validação prévia de exclusão.

📞 Mural no DOM (UC1) vs SQLite (UC2)

✗ Limitação do DOM (UC1):

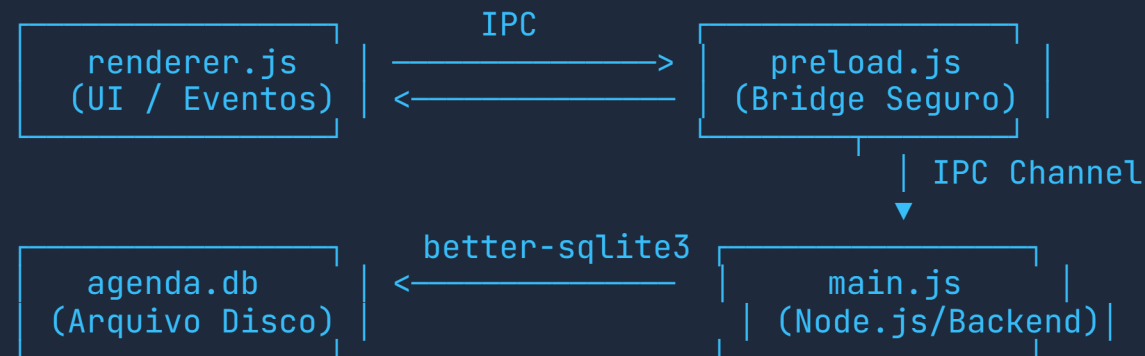
- Contatos armazenados no estado da página (`innerHTML += ...`).
- Ao recarregar (F5) ou fechar a janela, tudo é perdido (volatilidade da RAM).

✅ Solução Persistente no Electron (UC2):

- Dados gravados em arquivo físico no disco (`database/agenda.db`).
- Queries SQL garantem ordenação, busca eficiente e integridade relacional.



Arquitetura da Aplicação em 4 Camadas



- **Isolamento de Segurança:** O Frontend (**renderer.js**) não acessa o arquivo **.db** diretamente.



Modelagem da Tabela **contatos**

```
CREATE TABLE IF NOT EXISTS contatos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL,  
  foto_url TEXT,  
  telefone TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL  
);
```

- **id** : Garantia de exclusão e atualização sem ambiguidade.
- **foto_url** : Armazena a URL da imagem para exibição no card.
- **email UNIQUE** : Retorna erro amigável se tentar cadastrar e-mail duplicado.



Mapeamento dos Canais IPC

Canal	Método SQL	Descrição
get-contatos	<code>.prepare(...).all()</code>	Retorna todos os contatos por ordem alfabética (<code>ORDER BY nome ASC</code>).
add-contato	<code>.prepare(...).run(...)</code>	Insere novo contato no banco { nome, foto_url, telefone, email } .
delete-contato	<code>.prepare(...).run(id)</code>	Exclui fisicamente o contato pelo <code>id</code> .

Sanitização com Regex & Confirmação

1. Limpeza do Telefone com Expressão Regular (Regex):

```
const tellimpo = inputTelefone.value.replace(/\D/g, '');  
// Exemplo: "(81) 99999-8888" → "81999998888"
```

- `\D`: Seleciona qualquer caractere que **NÃO** seja dígito numérico.
- `/g`: Flag global para substituir todas as ocorrências.

2. Confirmação Visual Pré-Deleção:

```
if (confirm(`Deseja realmente excluir o contato ${nome}?`)) {  
  await window.api.deleteContato(id);  
}
```


Roteiro da Prática Guiada (Code-Along)

1. Passo 1 (`conexao.js`): Configurar o `better-sqlite3` e executar a DDL `CREATE TABLE` .
2. Passo 2 (`main.js`): Registrar os 3 handlers `ipcMain.handle` com tratamento de erro.
3. Passo 3 (`preload.js`): Expor as funções em `window.api` .
4. Passo 4 (`renderer.js`): Conectar os formulários e botões de exclusão, executando `atualizarMural()` .

Desafio Prático & Próximos Passos

Desafio da Aula (Mural Avançado):

1. **Filtro em Tempo Real (LIKE):** Adicione um campo `<input id="busca">` e crie a rota `search-contatos` (`WHERE nome LIKE ?`).
2. **Validação de E-mail:** Trate o erro de e-mail duplicado exibindo uma mensagem amigável ao usuário.

Próxima Aula:

- **Tópico 06:** Desafio Kanban Persistente (Arrastar e soltar com persistência de status no SQLite).